Thread Continuation

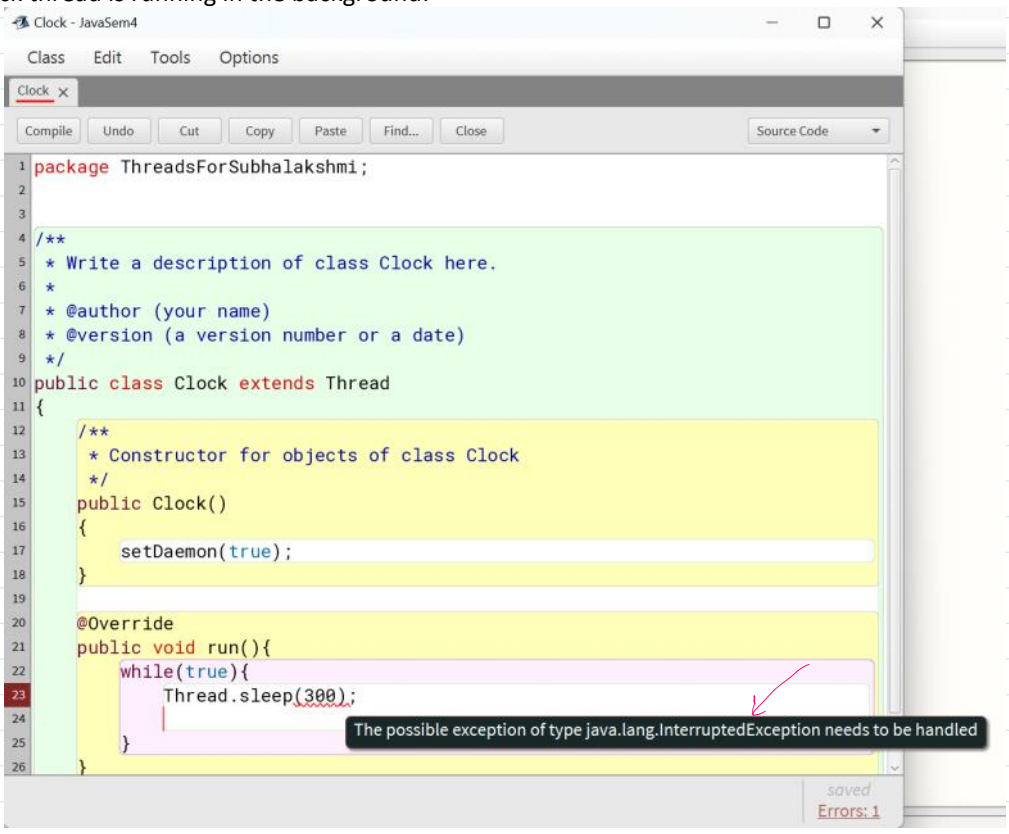
14 April 2024 09:44

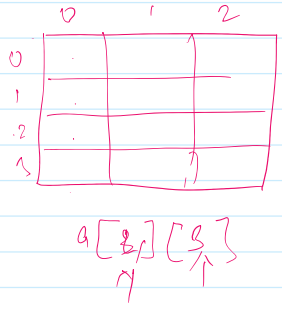
Two types of thread exist:

- User threads: this type of thread's activity is time-restricted, meaning its scenario is a process which ends after a certain amount of time. The JVM functions as long as there are user threads being executed.
- Demon threads: Threads that execute in the background as long as. the program is running. Demonstrates are only there to serve user threads. The JVM stops if there are only demons.
  Examples: syntax coloring in editors, garbage collectors etc.

- public final boolean isDaemon() in the thread class to determine the thread's type.
- public final void setDemon(boolean) of the Thread class.
  Marks this thread as either a daemon thread or a user thread.
- It must be called before the thread starts using the start() command.
- Use setDemon() in the constructor.

Clock thread is running in the background:





```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class Clock here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Clock extends Thread
{
   /**
```

```
     * Constructor for objects of class Clock
     */
    public Clock()
    {
        setDaemon(true);
    }

    @Override
    public void run(){
        while(true){
            try{
                Thread.sleep(300);
            }
            catch(InterruptedException e){

            }
            System.out.print("\ntick");
        }
    }
    public static void main(String args[]){
        System.out.print("\f");
        Clock ck = new Clock();
        ck.start();
    }
}
```

Monotask versus multitask

To illustrate multitask, let us begin by presenting a monotask execution. In the following example, a parrot (parrot.java) converses in monotask with the primary program(ChatAndLaunchTheParrot0.java):

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class Parrot here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Parrot
{
    private String cri;
    private int fois;
    public Parrot(String s, int i){
        cri = s;
        fois = i;
    }
    public void run(){
        int n;
        for(n = 0; n < fois; n++){
            try
            {
                Thread.sleep(1000);
            }
            catch (InterruptedException ie)
            {
```

```
            System.exit(1);
        }
        System.out.print(cri);
    }
  }
}
```

$O(n^2)$

Merge sort or quick sort → $O(n \log n)$

↑

close to linear

$O(n)$.

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class ChatAndLaunchTheParrot here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ChatAndLaunchTheParrot
{
   public static void main(String args[]){
      Parrot parrot = new Parrot("RAMA", 10);
      blabla(); blabla();
      parrot.run();
      for(int i = 0; i < 3; i++){
         try
         {
            Thread.sleep(1000);
         }
         catch (InterruptedException ie)
         {
            System.exit(1);
         }
         blabla();
      }
   }
   public static void blabla(){
      System.out.print("\nblabla");
   }
}
```

To perform the multi-task, a modification of the previous code is necessary. We will perform this in two different ways:

-Extending the Thread class

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class Parrot here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Parrot extends Thread
{
   private String cri;
   private int fois;
   public Parrot(String s, int i){
      cri = s;
```

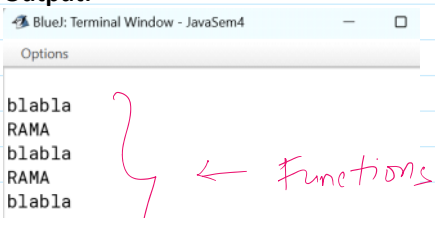
```
        fois = i;
    }
    @Override
    public void run(){
        int n;
        for(n = 0; n < fois; n++){
            try
            {
                Thread.sleep(1000);
            }
            catch (InterruptedException ie)
            {
                System.exit(1);
            }
            System.out.print("\n"+cri);
        }
    }
}

package ThreadsForSubhalakshmi;


/**
 * Write a description of class ChatAndLaunchTheParrot here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ChatAndLaunchTheParrot
{
    public static void main(String args[]){
        Parrot parrot = new Parrot("RAMA", 10);
        //blabla(); blabla();
        parrot.start();
        for(int i = 0; i < 10; i++){
            try
            {
                Thread.sleep(1000);
            }
            catch (InterruptedException ie)
            {
                System.exit(1);
            }
            blabla();
        }
    }
    public static void blabla(){
        System.out.print("\nblabla");
    }
}
```

start method calls the run() method implicitly.

Output:



← Functions executing in parallel.

```
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
```

← Functions executing in parallel.
Output is not in our control.

Can only enter input while your progra

⇒ Implementing Runnable interface.

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class P here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Parrot1 implements Runnable
{
   private String cri;
   private int fois;
   public Parrot1(String s, int i){
      cri = s;
      fois = i;
   }
   @Override
   public void run(){
      int n;
      for(n = 0; n < fois; n++){
         try
         {
            Thread.sleep(1000);
         }
         catch (InterruptedException ie)
         {
            System.exit(1);
         }
         System.out.print("\n"+cri);
      }
   }
}
```

← is the method to redefine in order to place the Thread's function.

```
package ThreadsForSubhalakshmi;


/**
```

```
 * Write a description of class ChatAndLaunchTheParrot here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ChatAndLaunchTheParrot
{
   public static void main(String args[]){
      //Parrot parrot = new Parrot("RAMA", 10);
      //blabla(); blabla();
      Parrot1 parrot = new Parrot1("RAMA", 10);
      Thread tparrot = new Thread(parrot);
      tparrot.start();
      for(int i = 0; i < 10; i++){
         try
         {
            Thread.sleep(1000);
         }
         catch (InterruptedException ie)
         {
            System.exit(1);
         }
         blabla();
      }
   }
   public static void blabla(){
      System.out.print("\nblabla");
   }
}
```

is a method that authorizes the Thread to initialize, and therefore the execution of run(). All it does is initialize the Thread and does not wait for run to finish execution.

It immediately returns to the execution of the calling Thread. The run() method of the Thread executes at the same time as the other Threads (multitask).

start() authorizes the Thread to start, but not necessarily immediately.

Output:

BlueJ: Terminal Window - JavaSem4

Options

```
blabla
RAMA
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
RAMA
blabla
```

Can only enter input while y

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class Runner here.
 *
```

```
 * @author (your name)
 * @version (a version number or a date)
 */
public class Runner extends Thread
{
   public Runner(String str){
      super(str);
   }

   @Override
   public void run(){
      int i;
      for(i = 1; i <= 10; i++){
         System.out.print("\n"+(i*100) + " m : " + getName());
         try
         {
            sleep((int)(Math.random() * 1000));
         }
         catch (InterruptedException ie)
         {
            ie.printStackTrace();
         }
      }
   }
}

package ThreadsForSubhalakshmi;


/**
 * Write a description of class Race here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Race
{
   public static void main(String args[]){
      System.out.print("\fPassing:");
      Runner Jean = new Runner("Jean");
      Runner Paul = new Runner("Paul");
      Jean.start();
      Paul.start();
   }
}
```

```
BlueJ: Terminal Window - JavaSem4
Options
Passing:
100 m : Paul
100 m : Jean
200 m : Jean
200 m : Paul
300 m : Jean
300 m : Paul
400 m : Jean
400 m : Paul
500 m : Paul
500 m : Jean
600 m : Paul
700 m : Paul
600 m : Jean
800 m : Paul
900 m : Paul
700 m : Jean
1000 m : Paul
800 m : Jean
900 m : Jean
1000 m : Jean

Can only enter input whil
```

Different states of a thread

A Thread can be in one of the four following states:

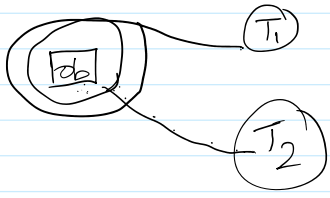
- New: Includes all moments between when it is created (by a constructor) and call for its start() method.
- Runnable: Immediately after the start method is called.
- Blocked: Representing the moments where the thread is waiting. For example, when it is suspended by the sleep(long) method, which pauses the execution of the thread for the given amount of time.
- Terminated: when its run() method ends.

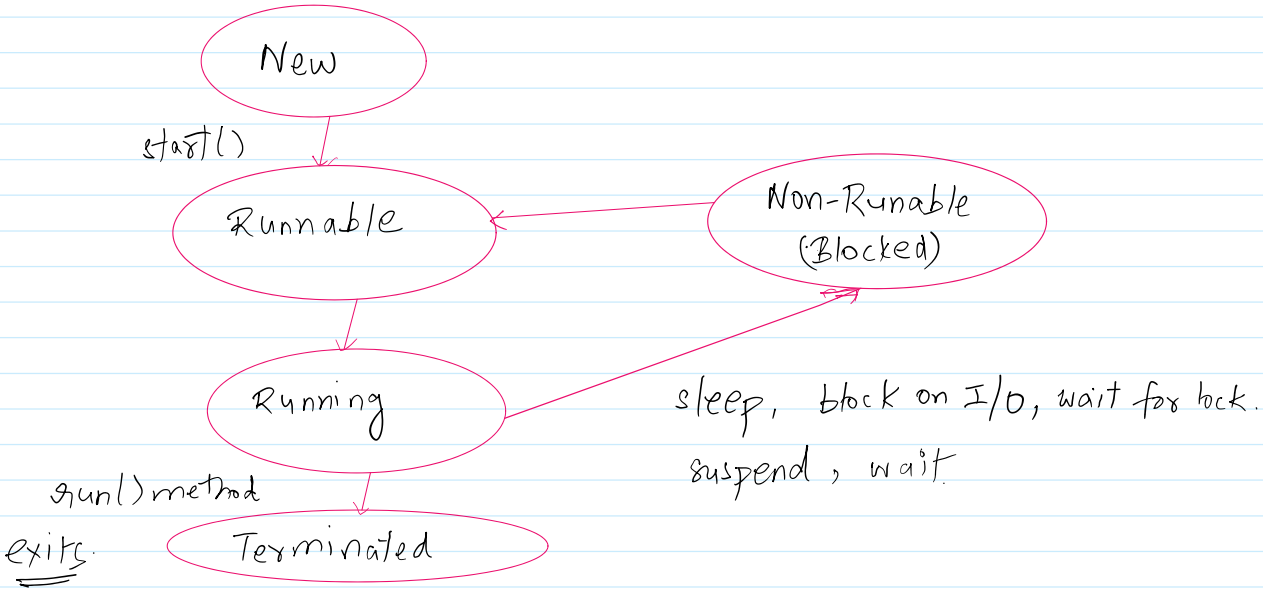
The state of the thread can be specified by the call for its getState() method.

Lifecycle of a thread

The lifecycle of a thread illustrates that it can be suspended by:

- Waiting for the end of the sleep.
- Waiting for an entry-exit block to end.
- Waiting for a wait synchronization





Life Cycle of a Thread.

For the creation of threads in Java, we have eight constructors

Constructor and Description

Thread()
Allocates a new Thread object.

Thread(Runnable target)
Allocates a new Thread object.

Thread(Runnable target, String name)
Allocates a new Thread object.

Thread(String name)
Allocates a new Thread object.

Thread(ThreadGroup group, Runnable target)
Allocates a new Thread object.

Thread(ThreadGroup group, Runnable target, String name)
Allocates a new Thread object so that it has target as its run object, has the specified name as its name, and belongs to the thread group referred to by group.

Thread(ThreadGroup group, Runnable target, String name, long stackSize)
Allocates a new Thread object so that it has target as its run object, has the specified name as its name, and belongs to the thread group referred to by group, and has the specified *stack size*.

Thread(ThreadGroup group, String name)
Allocates a new Thread object.

NOTE: Numerous threats can execute simultaneously It would be useful to be able to control them as one single entity to suspend them, to stop them, etc.

Java offers this possibility to higher the use of Thread groups:
java.lang.ThreadGroup

- We group a named set of threads
- They are controlled as one single unit.
- The JVM creates a minimum of one group of threads named the main.
- By default, a thread belongs to the same group as the one that create it(its parent);
- getThreadGroup(): to know its thread group.

- The class ThreadGroup has two constructors:
  - ThreadGroup(String name);
  - ThreadGroup(ThreadGroup parent, String name);

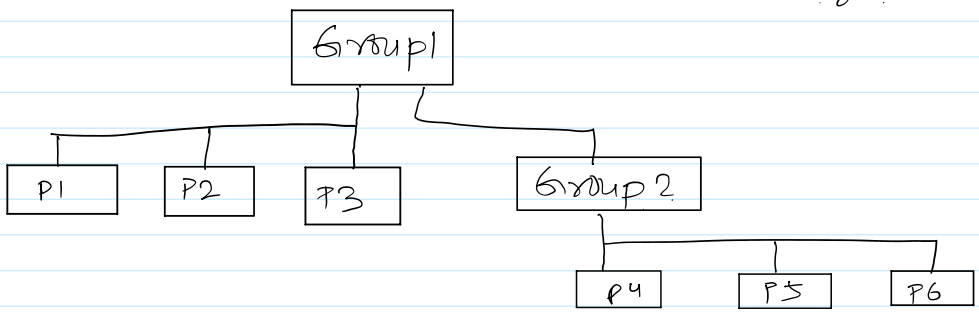
```
ThreadGroup tg = new ThreadGroup("Group1");
Thread p1 = new Thread(tg, "p1");
Thread p2 = new Thread(tg, "p2");
Thread p3 = new Thread(tg, "p3");
```

Control of the ThreadGroup goes through the use of standard methods that are shared with thread: interrupt(), destroy().
For example: applying the interrupt() method to "Group1" means invoking this method for each thread within the group("p1", "p2" and "p3").

```
//1st group
ThreadGroup tg1 = new ThreadGroup("Group1");
Thread p1 = new Thread(tg1, "p1");
Thread p2 = new Thread(tg1, "p2");
Thread p3 = new Thread(tg1, "p3");
//2nd group
ThreadGroup tg2 = new ThreadGroup(tg1,"Group2");
Thread p4 = new Thread(tg2, "p1");
Thread p5 = new Thread(tg2, "p2");
Thread p6 = new Thread(tg2, "p3");
```

```
ThreadGroup tg2 = new ThreadGroup(tg1, "Group2");
Thread p4 = new Thread(tg2, "p1");
Thread p5 = new Thread(tg2, "p2");
Thread p6 = new Thread(tg2, "p3");
```

tg2.interrupt();



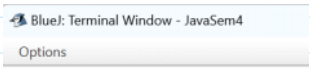
Thread tree

```
public class Parrot2 extends Thread
{
   private String cri;
   private int fois;
   public Parrot2(String s, int i){
      cri = s;
      fois = i;
   }
   @Override
   public void run(){
      int n;
      for(n = 0; n < fois; n++){
         repeater();
      }
   }
   public void repeater(){
      System.out.print("\n"+cri);
   }
}

public class ChatAndLaunchTheParrot2
{
   public static void main(String args[]){
      //Parrot parrot = new Parrot("RAMA", 10);
      //blabla(); blabla();
      Parrot2 parrot2 = new Parrot2("RAMA", 10);
      Thread tparrot = new Thread(parrot2);
      tparrot.start();
      for(int i = 0; i < 10; i++){
         blabla();
      }
   }

   public static void blabla(){
      System.out.print("\nblabla");
   }
}
```

We were launching two threads without using sleep, the primary program and another thread.



This is clearly a multitask execution but the execution time is too short to visualize the allocation of the CPU

```
BlueJ: Terminal Window - JavaSem4
Options

blabla
blabla
blabla
blabla
blabla
blabla
blabla
blabla
blabla
blabla
RAMA
RAMA
RAMA
RAMA
RAMA
RAMA
RAMA
RAMA
RAMA
RAMA

Can only enter input while
```

This is clearly a multitask execution but the execution time is too short to visualize the allocation of the cpu between the two threads

Time allocation between two threads.

In order to visualize the allocation of time between true threads, we will. have to increase the number of iterations to 10. to 10 million by examining which thread executes in multiple of 100,000.

```
package ThreadsForSubhalakshmi;


/**
 * Write a description of class Runner1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Runner1 extends Thread
{
   String nm;
   public Runner1(String name){
      super(name);
      this.nm = name;
   }

   @Override
   public void run(){
      long pedalStroke = 0;
      while(pedalStroke < 1000000){
         pedalStroke++;
         if(pedalStroke % 500 == 0){
            System.out.print("\nRunner " + nm + " performs " +
            pedalStroke+" pedal strokes.");
         }
      }
   }
}

package ThreadsForSubhalakshmi;
```

```
/**
 * Write a description of class InfernalRace here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class InfernalRace
{
   public static void main(String args[]){
      System.out.print("\f");
      Runner1 A = new Runner1("Kevin");
      Runner1 B = new Runner1("Jack");
      A.start();
      B.start();
   }
}
```

Conclusion:- Java does not require that the system be "time sliced", the same amount of time is given to threads with the same level of priority.

Priority between threads

The setPriority sets a level of priority between the different threads. The value must be between a minimum value, MIN_PRIORITY, and a maximum one, MAX_PRIORITY.

package ThreadsForSubhalakshmi;

```
/**
 * Write a description of class Runner1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Runner1 extends Thread
{
   String nm;
   public Runner1(String name){
      super(name);
      this.nm = name;
   }

   @Override
   public void run(){
      long pedalStroke = 0;
      while(pedalStroke < 1000000){
         pedalStroke++;
         if(pedalStroke % 50000 == 0){
            System.out.print("\nRunner " + nm + " performs " +
            pedalStroke+" pedal strokes.");
         }
      }
   }
}

public class InfernalRace
{
   public static void main(String args[]){
      System.out.print("\f");
      Runner1 A = new Runner1("Kevin");
      Runner1 B = new Runner1("Jack");
```

```
    A.setPriority(Thread.MIN_PRIORITY);
    B.setPriority(Thread.MAX_PRIORITY);
    System.out.print("\nThread runner " + A.nm + " has priority = "+
    A.getPriority());
    System.out.print("\nThread runner " + B.nm + " has priority = "+
    B.getPriority());

    A.start();
    B.start();
  }
}
```

Thread B accessed the CPU first because it had maximum priority. Thread A executed at the end of Thread A because it has minimum priority. Priority allowed us to define an order of execution between two threads.

Output:

```
Thread runner Kevin has priority = 1
Thread runner Jack has priority = 10
Runner Jack performs 50000 pedal strokes.
Runner Kevin performs 50000 pedal strokes.
Runner Jack performs 100000 pedal strokes.
Runner Jack performs 150000 pedal strokes.
Runner Kevin performs 100000 pedal strokes.
Runner Jack performs 200000 pedal strokes.
Runner Kevin performs 150000 pedal strokes.
Runner Jack performs 250000 pedal strokes.
Runner Jack performs 300000 pedal strokes.
Runner Jack performs 350000 pedal strokes.
Runner Kevin performs 200000 pedal strokes.
Runner Jack performs 400000 pedal strokes.
Runner Kevin performs 250000 pedal strokes.
Runner Jack performs 450000 pedal strokes.
Runner Jack performs 500000 pedal strokes.
Runner Kevin performs 300000 pedal strokes.
Runner Jack performs 550000 pedal strokes.
Runner Kevin performs 350000 pedal strokes.
Runner Jack performs 600000 pedal strokes.
Runner Kevin performs 400000 pedal strokes.
Runner Kevin performs 450000 pedal strokes.
Runner Jack performs 650000 pedal strokes.
Runner Kevin performs 500000 pedal strokes.
Runner Jack performs 700000 pedal strokes.
Runner Kevin performs 550000 pedal strokes.
Runner Jack performs 750000 pedal strokes.
Runner Kevin performs 600000 pedal strokes.
Runner Jack performs 800000 pedal strokes.
Runner Kevin performs 650000 pedal strokes.
Runner Jack performs 850000 pedal strokes.
Runner Kevin performs 700000 pedal strokes.
Runner Kevin performs 750000 pedal strokes.
Runner Jack performs 900000 pedal strokes.
Runner Kevin performs 800000 pedal strokes.
Runner Jack performs 950000 pedal strokes.
Runner Kevin performs 850000 pedal strokes.
Runner Kevin performs 900000 pedal strokes.
Runner Jack performs 1000000 pedal strokes.
Runner Kevin performs 950000 pedal strokes.
Runner Kevin performs 1000000 pedal strokes.
```

- Priority between threads: method setPriority(int)
- The priorities of thread are between 1 and 10.
- Three constant represents the limit and median values.
- higher the value, higher the priority of thread to access the processor.
  - MAX_PRIORITY → 10
  - NORM_PRIORITY → 5
  - MIN_PRIORITY → 1

Normal priority is assigned by default to a new thread.